Multi-Source Candidate Data Transformer - Technical Design

Candidate: ***Anwita Chakraborty*** | *Email:* ***anwita795@gmail.com*** | *Date: July 2026*

1. System Architecture & Pipeline Breakdown

The transformer operates as a deterministic multi-stage data processing pipeline. Separation of concerns is maintained between data ingestion, deduplication/merging, and custom schema projection:

* **Ingestion (Stage 1):** Independent, low-coupling source modules ingest files (structured and unstructured). Missing directories or garbage source formats (such as invalid JSON) are logged and skipped without halting the system.
* **Normalization (Stage 2):** Low-level sanitization: phone numbers coerced to E.164; dates parsed and normalized to YYYY-MM; location strings split; skill names mapped to canonical representations using aliases.
* **Merging & Deduplication (Stage 3):** Records are grouped by unique identity keys. Within-source duplicates are resolved, and cross-source fields are unified based on trust tiers. All modifications and overrides populate a provenance history.
* **Projection & Validation (Stage 4):** Runtime configuration maps canonical records into targeted outputs. Fields are optionally filtered, remapped, normalized, or discarded (omit/null/error on missing) and validated against a schema.

2. Canonical Internal Schema & Normalizations

The pipeline standardizes candidate information into a Pydantic-based *CandidateRecord* representation:

* **Dates:** Normalized to ISO YYYY-MM using regex heuristics, mapping short months (Jan) and full months (June).
* **Phones:** Coerced to E.164 format (using +91 or +1 country prefixes, removing formatting spaces/dashes).
* **Locations:** Parsed into a structured *LocationRecord* containing city, region, and ISO-3166 alpha-2 country codes.
* **Skills:** Coerced to lowercase and canonicalized using an alias table (such as ReactJS/React.js to react, Node.js to node).

3. Identity Matching & Conflict-Resolution Policy

* **Identity Keys:** Candidates are resolved using *email* as the primary matching key. A fallback match key of *name + normalized-phone* is applied to merge records lacking an email (such as resume or recruiter notes matches). Near-duplicates (like Aman vs Amann Sharma) are protected against incorrect merges.
* **Trust-Tier Hierarchy:** Out-of-source conflicts are resolved using a predefined trust hierarchy rank: **Resume (5) > ATS JSON (4) > Recruiter CSV (3) > LinkedIn/GitHub (2) > Recruiter Notes (1)**. The highest-ranked source wins. For equal-trust conflicts, the most recently updated record (highest last_updated date) is chosen.
* **Alternatives Logging:** Overridden or suppressed values are stored in the *alternatives* dictionary, documenting the value, source name, trust score, and suppression reason (such as lower_trust, lower_recency, or in_source_duplicate_older).
* **In-Source Deduplication:** When duplicate rows appear in a single source, the most recent timestamp wins. The older values are suppressed from the active profile and registered as alternatives.

4. Runtime Schema Projection & Validation

The system accepts a runtime configuration mapping that transforms canonical records without altering codebase rules:

* **Paths:** Resolves nested attributes (like *location.city*), array indices (like *emails[0]*), and collections (like *skills[].name*).
* **Missing Policy:** Handles missing data gracefully according to the configuration's *on_missing* behavior: **null** (keeps field as null), **omit** (removes key from output), or **error** (raises a descriptive ValueError).
* **Validation:** Projects the configuration mapping and validates the result against the target schema types (string, number, object, string[], object[]) to guarantee structural safety before returning.

5. Critical Edge Cases & Scope Trade-offs under Time Pressure

* **Handled Edge Cases:** (1) Semi-structured JSON truncation (C09) and empty resumes (C14) are handled without crashes; (2) Candidates matching via fallbacks without email (C06, C16) merge correctly; (3) In-source duplicates (C13) resolve phone conflicts while preserving the older number in alternatives.
* **Time-Pressure Trade-offs:** To prioritize core correctness, we chose: (1) Regex-based heuristic parser for resumes rather than heavy NLP or API-based LLMs; (2) Simple canonical skill mapping instead of building an extensive skill taxonomy engine; (3) Excluded multi-email resolution (candidates using personal and work emails are treated as separate profiles instead of cross-linked).